```
void wait(std::unique_lock<std::mutex>& lock);
```

Preconditions

`lock.owns_lock()` is `true`, and the lock is owned by the calling thread.

Effects

Atomically unlocks the supplied `lock` object and block until the thread is woken by a call to `notify_one()` or `notify_all()` by another thread, or the thread is woken spuriously. The `lock` object is locked again before the call to `wait()` returns.

Throws

`std::system_error` if the effects can't be achieved. If the `lock` object is unlocked during the call to `wait()`, it's locked again on exit, even if the function exits via an exception.

Note

The spurious wakeups mean that a thread calling `wait()` may wake even though no thread has called `notify_one()` or `notify_all()`. It's therefore recommended that the overload of `wait()` that takes a predicate is used in preference where possible. Otherwise, it's recommended that `wait()` be called in a loop that tests the predicate associated with the condition variable.

Synchronization

Calls to `notify_one()`, `notify_all()`, `wait()`, `wait_for()`, and `wait_until()` on a single